

SUPMEAL

Alternative libre à Mealie et Tandoor Recipes

Documentation technique

Architecture, choix technologiques, modélisation UML,
schéma de base de données et guide de déploiement

Auteur	Paul Mandin
Établissement	SUPINFO
Formation	Master 1 — Session de rattrapage (4RESIT)
Projet	SUPMEAL Pro
Version du document	1.0
Date	Août 2026

Ce document est destiné à un public technique : développeurs, DevOps et équipes chargées de la maintenance ou de l'évolution de l'application.

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectif du document	3
1.3	Vue d'ensemble fonctionnelle	3
2	Architecture générale	5
2.1	Architecture trois-tiers	5
2.2	Architecture en couches du serveur	5
2.3	Communication temps réel	6
2.4	Stockage des fichiers	6
3	Choix technologiques et justifications	7
3.1	Vue d'ensemble de la stack	7
3.2	Justification des choix	7
3.2.1	Node.js et Express.js	7
3.2.2	Sequelize et PostgreSQL	7
3.2.3	JavaScript vanilla côté client	8
3.2.4	JWT plutôt que sessions serveur	8
3.2.5	bcryptjs	8
3.2.6	Google OAuth2 (google-auth-library)	8
3.2.7	Socket.io	8
3.2.8	Multer	9
3.2.9	fuse.js	9
3.2.10	Docker et Docker Compose	9
4	Modélisation UML	10
4.1	Diagramme de cas d'utilisation	10
4.2	Diagramme de classes	11
5	Schéma de la base de données	13
5.1	Vue d'ensemble	13
5.2	Description des tables	14
5.3	Intégrité référentielle	14
5.4	Index et recherche	15
6	Vue d'ensemble de l'API REST	16
6.1	Conventions générales	16
6.2	Vue d'ensemble des ressources	16

6.3	Authentification — <code>/api/auth</code>	17
6.4	Sous-ressources et actions spécifiques	17
6.5	Filtrage et recherche des recettes	18
6.6	Export et import — <code>/api/export</code> , <code>/api/import</code>	18
6.7	Communication temps réel — <code>Socket.io</code>	19
7	Informations nécessaires au fonctionnement	20
7.1	Variables d’environnement	20
7.2	Prérequis pour activer la connexion Google	21
7.3	Prérequis logiciels	21
7.4	Secrets et rendu du projet	21
8	Guide de déploiement	22
8.1	Déploiement via Docker Compose (recommandé)	22
8.1.1	Étapes	22
8.1.2	Détail des services	22
8.1.3	Arrêt et nettoyage	23
8.2	Déploiement en environnement de développement local (sans Docker)	23
8.2.1	Base de données	23
8.2.2	Serveur	23
8.2.3	Client	23
8.3	Vérification post-déploiement	24
9	Sécurité	25
9.1	Authentification et gestion des secrets	25
9.2	Protection HTTP	25
9.3	Contrôle d’accès et permissions	25
9.4	Validation des entrées	26
9.5	Upload de fichiers	26
9.6	Isolation réseau	26

Chapitre 1

Introduction

1.1 Contexte du projet

La société fictive **SUPMEAL Pro** souhaite fournir à ses employés et à ses clients un outil de gestion de recettes et de planification de repas. **SUPMEAL** se positionne comme une alternative complète et intuitive à des solutions existantes telles que Mealie, Tandoor Recipes ou Paprika, souvent limitées dans leur version gratuite ou payantes.

L'application permet à chaque utilisateur de gérer ses recettes personnelles, de créer des *cookbooks* (livres de recettes) partagés avec d'autres membres, de planifier des repas, d'échanger via une messagerie intégrée et d'exporter/importer ses données.

1.2 Objectif du document

Cette documentation technique s'adresse à un public de professionnels du développement logiciel (développeurs, DevOps, mainteneurs). Elle a pour objectif de :

- présenter l'architecture générale de l'application et les choix technologiques effectués, avec leur justification ;
- fournir la modélisation UML du système (cas d'utilisation, classes) ;
- décrire le schéma de la base de données relationnelle ;
- lister les informations et variables nécessaires au bon fonctionnement de l'application ;
- fournir un guide de déploiement complet, via Docker Compose ou en environnement de développement local ;
- décrire les mécanismes de sécurité mis en place.
- Le manuel utilisateur, destiné aux utilisateurs finaux de l'application, fait l'objet d'un document séparé.

1.3 Vue d'ensemble fonctionnelle

SUPMEAL couvre les fonctionnalités suivantes :

- **Authentification** : inscription/connexion classique (email + mot de passe) et connexion via Google OAuth2.
- **Cookbooks partagés** : création, invitation de membres, gestion fine des rôles (Créateur, Éditeur, Lecteur, Commentateur).

- **Gestion des recettes** : titre, ingrédients structurés, étapes, temps de préparation/cuisson, portions, tags, image, source, favoris, planification.
- **Filtrage et recherche** : par cookbook, tags, ingrédients, temps, favoris, ainsi qu'une recherche plein texte.
- **Préférences utilisateur** : régime alimentaire, allergies, cuisines préférées, portions par défaut.
- **Export / Import** : format JSON et CSV.
- **Messagerie et commentaires** : discussion instantanée par cookbook (WebSocket) et commentaires par recette.

Chapitre 2

Architecture générale

2.1 Architecture trois-tiers

Conformément au cahier des charges, SUPMEAL est découpé en trois briques indépendantes, chacune exécutée dans son propre conteneur Docker :

- un **client web** statique (HTML/CSS/JavaScript vanilla), servi par Nginx, qui n'implémente **aucune logique métier** : il ne fait qu'appeler l'API REST et le WebSocket, puis restituer les réponses ;
- un **serveur API REST** (Node.js/Express) qui centralise l'intégralité de la logique métier, des règles de validation et des permissions ;
- une **base de données** relationnelle PostgreSQL, accessible uniquement depuis le serveur.

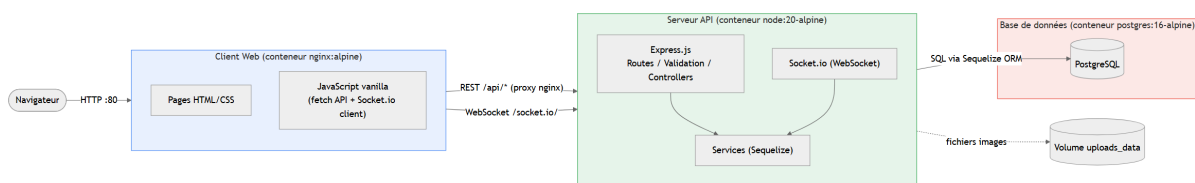


FIGURE 2.1 – Architecture trois-tiers de SUPMEAL et flux réseau entre conteneurs

Le client ne dialogue jamais directement avec la base de données : Nginx reverse-proxy les requêtes `/api/*`, `/uploads/*` et `/socket.io/*` vers le conteneur serveur (voir `client/nginx.conf`), ce qui évite d'exposer le port du serveur applicatif directement au navigateur et centralise l'origine servie au client sur le port 80.

2.2 Architecture en couches du serveur

Le serveur suit une architecture en couches strictement séparées, qui limite le couplage et facilite les tests et la maintenance :

Couche	Rôle
<code>routes/*.routes.js</code>	Déclaration des endpoints et chaînage des middlewares

Couche	Rôle
<code>middleware/*.validation.js</code>	Validation des entrées (express-validator)
<code>middleware/validateRequest.js</code>	Centralisation du traitement des erreurs de validation
<code>middleware/*.js</code> (auth, rôles)	Authentification et contrôle d'accès (permissions)
<code>controllers/*.controller.js</code>	Orchestration : reçoit la requête, appelle les services, formate la réponse
<code>services/*.services.js</code>	Logique métier et accès aux données via Sequelize
<code>models/*.js</code>	Définition du schéma de données et des associations Sequelize

Chaque requête HTTP traverse ainsi une chaîne bien définie :

Route → Validation → Gestion des erreurs de validation → Contrôle d'accès → Controller → Service → Modèle (Sequelize)

Les validateurs sont séparés **par domaine** (`auth.validation.js`, `cookbook.validation.js`, `recipe.validation.js`, etc.) plutôt que regroupés dans un fichier monolithique, ce qui garde chaque module aligné sur le découpage routes/controllers/services et facilite la localisation des règles de validation d'un domaine donné.

2.3 Communication temps réel

En complément de l'API REST, un serveur **Socket.io** est monté sur le même processus HTTP (`server/src/sockets/index.js`). Il gère :

- l'authentification de la connexion WebSocket via le token JWT transmis dans le handshake ;
- la jonction à une *room* par `cookbook` (`cookbook:{id}`), après vérification de l'appartenance de l'utilisateur ;
- la diffusion des nouveaux messages de la messagerie instantanée à tous les membres connectés d'un `cookbook`.

2.4 Stockage des fichiers

Les images de recettes sont stockées sur disque via **Multer** (stockage local) dans le répertoire `server/uploads`, monté comme volume Docker nommé (`uploads_data`) afin de survivre au redémarrage ou à la recréation du conteneur serveur. Elles sont servies statiquement par Express sous le chemin `/uploads`.

Chapitre 3

Choix technologiques et justifications

3.1 Vue d'ensemble de la stack

Composant	Technologie	Rôle
Backend	Node.js 20 + Express.js 4	API REST
ORM	Sequelize 6	Mapping objet-relationnel vers PostgreSQL
Base de données	PostgreSQL 16	Persistance relationnelle
Temps réel	Socket.io 4	Messagerie instantanée
Authentification	JWT (jsonwebtoken) + bcryptjs	Sessions sans état, hachage des mots de passe
OAuth2	google-auth-library	Connexion via Google
Upload	Multer	Upload d'images et de fichiers d'import
Frontend	HTML / CSS / JavaScript vanilla	Interface multi-pages (MPA)
Serveur web statique	Nginx (alpine)	Service du client et reverse proxy vers l'API
Conteneurisation	Docker / Docker Compose	Déploiement reproductible

3.2 Justification des choix

3.2.1 Node.js et Express.js

Express est un framework HTTP minimaliste, très largement adopté et documenté, qui laisse une totale liberté sur l'organisation du code. Il convient parfaitement à une API REST de taille moyenne sans impliquer la complexité (et l'empreinte mémoire) d'un framework plus opinioné comme NestJS. Son écosystème de middlewares (`cors`, `helmet`, `morgan`, `express-validator`, `express-rate-limit`) couvre l'ensemble des besoins transverses du projet (sécurité HTTP, journalisation, validation, limitation de débit) sans dépendance supplémentaire lourde.

3.2.2 Sequelize et PostgreSQL

Sequelize est un ORM mature pour Node.js, avec un support complet de PostgreSQL (types `ARRAY`, `ENUM`, `UUID`) et une API de définition des associations (`hasMany`, `belongsTo`, `belongsToMany`) qui correspond directement aux relations modélisées dans le schéma de données (voir chapitre 5). PostgreSQL a été choisi plutôt qu'une base NoSQL car le modèle de données de SUPMEAL est fortement relationnel (utilisateurs, cookbooks, recettes, ingrédients, tags, favoris, planification) avec de nombreuses relations many-to-many : un SGBD relationnel garantit l'intégrité référen-

tielle (contraintes de clé étrangère, `ON DELETE CASCADE`) nativement, sans la reconstruire côté application.

3.2.3 JavaScript vanilla côté client

Le client ne comporte volontairement aucun framework front-end (pas de React, Vue ou Vite). Ce choix découle directement de la contrainte du cahier des charges imposant l'absence de logique métier côté client : une architecture multi-pages (MPA) avec des appels `fetch()` explicites rend cette séparation évidente et vérifiable, sans la couche d'abstraction (state management, hydratation, build step) qu'introduirait un framework SPA. Cela simplifie également le déploiement : le client est un ensemble de fichiers statiques servis tels quels par Nginx, sans étape de compilation.

3.2.4 JWT plutôt que sessions serveur

L'authentification par **JSON Web Token** vérifié manuellement (sans Passport.js) a été préférée à des sessions serveur avec stockage (Redis, base de données) car elle ne nécessite aucun état partagé côté serveur : chaque requête porte sa propre preuve d'authentification (`Authorization: Bearer <token>`), ce qui simplifie la mise à l'échelle horizontale du serveur API et évite une dépendance supplémentaire (magasin de sessions). Le même token est réutilisé pour authentifier la connexion WebSocket (handshake Socket.io), évitant un second mécanisme d'authentification.

3.2.5 bcryptjs

Le hachage des mots de passe utilise `bcryptjs`, une implémentation pure JavaScript de `bcrypt` (facteur de coût 10), évitant toute dépendance à des binaires natifs compilés (contrairement à `bcrypt`), ce qui simplifie la construction de l'image Docker `node:20-alpine` et la rend portable sans outillage de compilation supplémentaire.

3.2.6 Google OAuth2 (google-auth-library)

Le sujet demande la possibilité de se connecter via un fournisseur OAuth2. Google a été retenu (plutôt que GitHub ou Microsoft) car c'est le fournisseur le plus universellement utilisé par les utilisateurs finaux visés (employés et clients d'une société), et `google-auth-library` offre une vérification côté serveur du jeton d'identité (`verifyIdToken`) sans nécessiter d'échange de secret client côté navigateur (flux *Google Identity Services* côté client, jeton vérifié côté serveur). L'architecture prévoit une table `OAuthAccount` générique permettant d'ajouter GitHub ou Microsoft sans modification du schéma.

3.2.7 Socket.io

Pour la messagerie instantanée et les mises à jour en temps réel des commentaires/messages, Socket.io a été préféré à une implémentation WebSocket native car il gère automatiquement la reconnexion, le fallback HTTP long-polling, et un système de *rooms* directement exploitable pour isoler la diffusion des messages par cookbook.

3.2.8 Multer

Multer est le middleware standard de l'écosystème Express pour gérer les upload multipart/form-data. Il est utilisé avec deux configurations distinctes : stockage disque avec filtrage MIME pour les images de recettes (limite 5 Mo), et stockage mémoire pour les fichiers d'import (JSON/CSV/Mealie, limite 10 Mo) qui sont traités à la volée sans persister le fichier brut sur disque.

3.2.9 fuse.js

La recherche plein texte (titre, description, ingrédients, tags) s'appuie sur `fuse.js`, une bibliothèque de recherche floue côté serveur, permettant de tolérer les fautes de frappe et les correspondances partielles sans nécessiter l'installation d'un moteur de recherche externe (Elasticsearch, Meilisearch) disproportionné pour le volume de données visé.

3.2.10 Docker et Docker Compose

La conteneurisation garantit un environnement d'exécution reproductible, indépendant du poste de développement, et respecte l'exigence du cahier des charges de fournir trois services Docker distincts (client, serveur, base de données) orchestrés par un unique `docker-compose.yml` à la racine du projet. Les images `node:20-alpine` et `postgres:16-alpine/nginx:alpine` minimisent la surface d'attaque et la taille des images produites.

Chapitre 4

Modélisation UML

4.1 Diagramme de cas d'utilisation

Trois niveaux d'acteurs interagissent avec le système : un **visiteur** non authentifié, un **utilisateur authentifié** et un **membre de cookbook** (utilisateur authentifié disposant d'un rôle au sein d'un cookbook partagé donné). La figure [4.1](#) synthétise les principaux cas d'utilisation par domaine fonctionnel.



FIGURE 4.1 – Diagramme de cas d'utilisation de SUPMEAL

4.2 Diagramme de classes

Le diagramme de classes ci-dessous reflète directement les modèles Sequelize définis dans `server/src/models/` ainsi que leurs associations (`server/src/models/index.js`). Les cardinalités indiquées sont celles imposées au niveau applicatif par Sequelize.

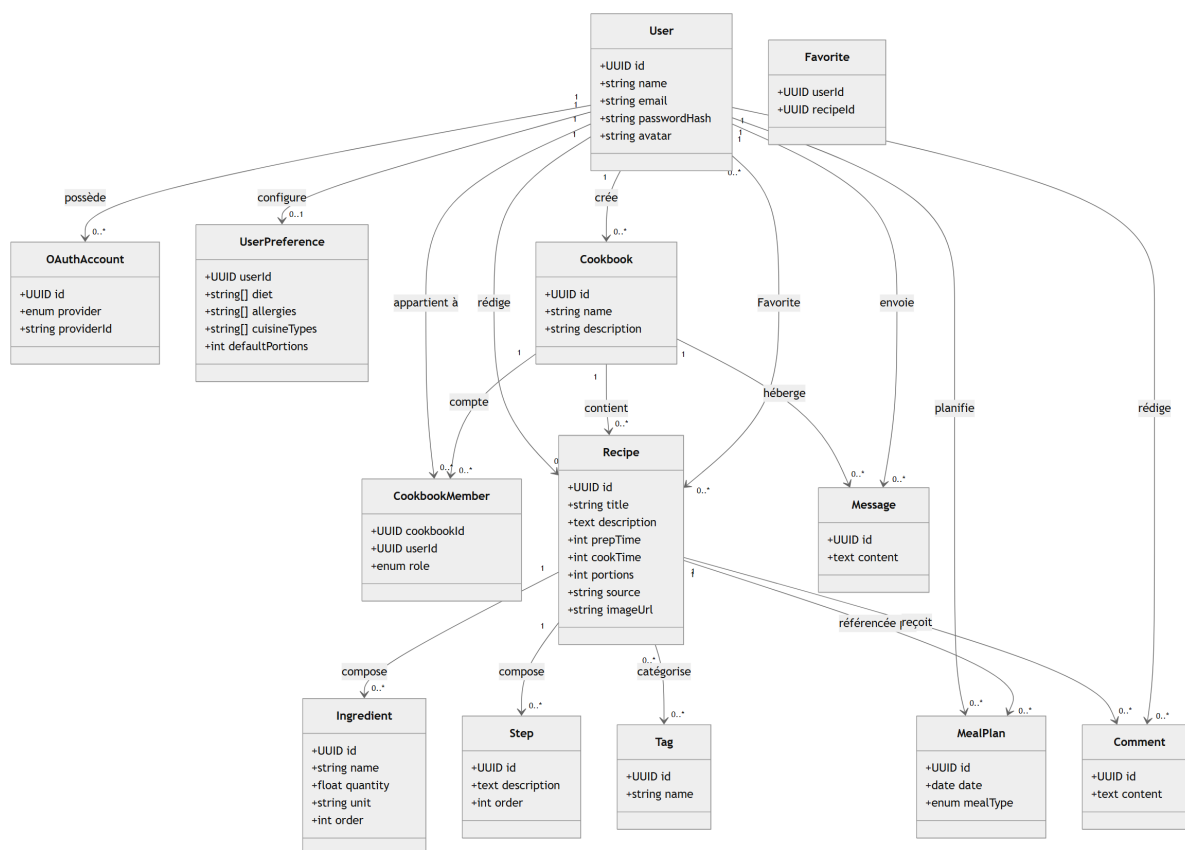


FIGURE 4.2 – Diagramme de classes du domaine SUPMEAL

Points notables du modèle :

- CookbookMember est une table d’association enrichie (clé primaire composée cookbookId+userId) portant l’attribut role, qui matérialise les quatre niveaux de permission demandés par le sujet : CREATOR, EDITOR, READER, COMMENTER.
- Recipe.cookbookId est nullable : une recette peut appartenir au compte personnel d’un utilisateur (cookbookId = NULL) ou à un cookbook partagé.
- Favorite est une table d’association pure (many-to-many entre User et Recipe) sans attribut métier supplémentaire.
- OAuthAccount est conçu pour supporter plusieurs fournisseurs (google, github, microsoft) via une contrainte d’unicité composée sur (provider, providerId), ce qui permet à un même User de lier plusieurs comptes OAuth.

Chapitre 5

Schéma de la base de données

5.1 Vue d'ensemble

La base de données PostgreSQL comporte 14 tables : 13 modèles Sequelize explicites (`server/src/models/`) et une table de jonction implicite (`RecipeTag`), générée automatiquement par Sequelize pour la relation many-to-many entre `Recipe` et `Tag`. Les identifiants primaires sont des UUID (générés côté application via `DataTypes.UUIDV4`) plutôt que des entiers auto-incrémentés, afin d'éviter toute énumération séquentielle des ressources par un client de l'API et de faciliter une éventuelle fusion de données importées. Le schéma est entièrement dérivé des modèles Sequelize (synchronisation via `sequelize.sync()` au démarrage du serveur).

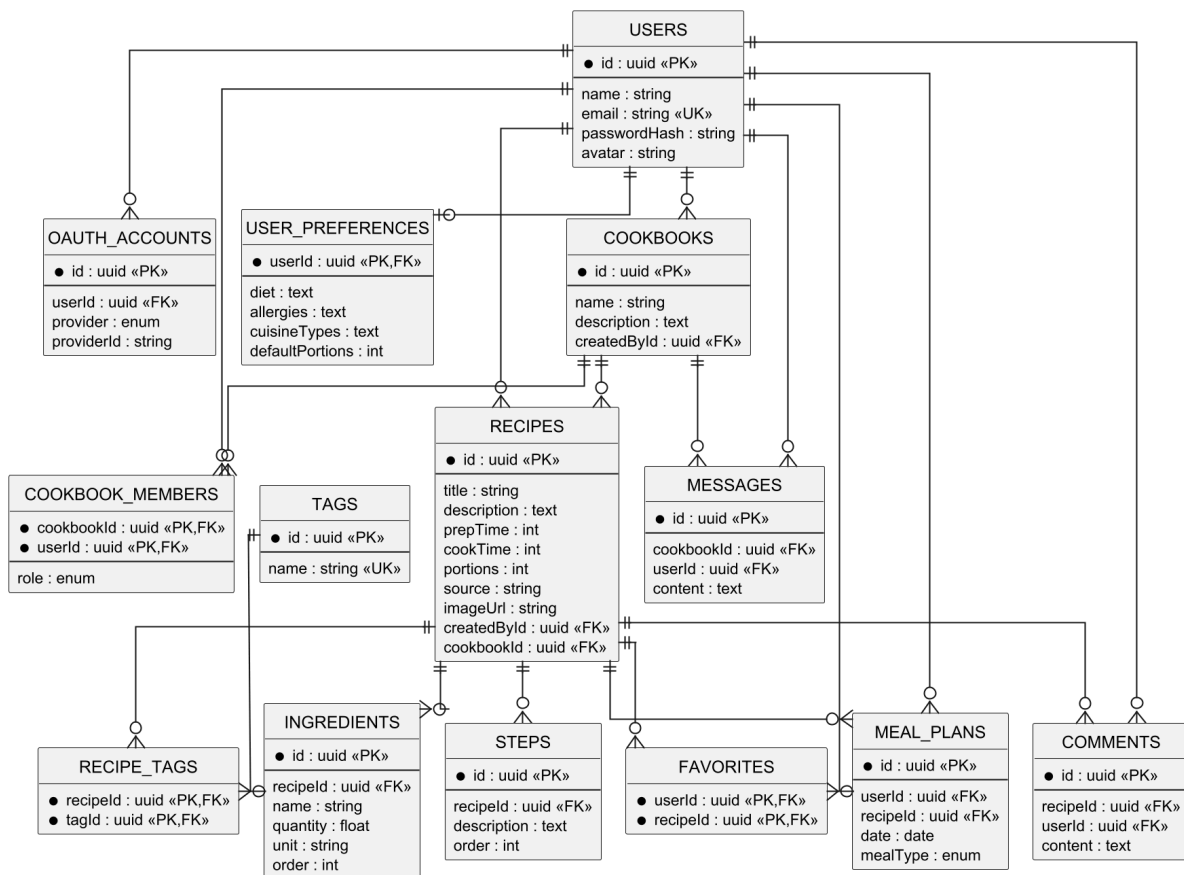


FIGURE 5.1 – Schéma entité-association de la base de données SUPMEAL

5.2 Description des tables

Table	Rôle
Users	Comptes utilisateurs (email/mot de passe haché et/ou lié à un fournisseur OAuth).
OAuthAccounts	Liaison entre un utilisateur et un compte tiers (Google, extensible à GitHub/Microsoft).
UserPreferences	Préférences culinaires (régime, allergies, cuisines préférées, portions par défaut) — relation 1 :1 avec Users.
Cookbooks	Livres de recettes partagés.
CookbookMembers	Table d'association Users×Cookbooks portant le rôle du membre.
Recipes	Recettes, personnelles (cookbookId NULL) ou appartenant à un cookbook.
Ingredients	Ingrédients structurés d'une recette (nom, quantité, unité, ordre d'affichage).
Steps	Étapes de préparation ordonnées d'une recette.
Tags	Catégories/tags réutilisables (type de cuisine, régime, difficulté...).
RecipeTags	Table d'association Recipes×Tags.
Favorites	Table d'association Users×Recipes pour les recettes favorites.
MealPlans	Planification d'une recette à une date et un type de repas donnés pour un utilisateur.
Comments	Commentaires d'un utilisateur sur une recette.
Messages	Messages de la messagerie instantanée d'un cookbook.

5.3 Intégrité référentielle

Les contraintes de suppression en cascade sont définies explicitement au niveau des associations Sequelize afin de garantir la cohérence des données :

- la suppression d'un User entraîne la suppression en cascade (ON DELETE CASCADE) de ses OAuthAccounts, UserPreferences, CookbookMembers, Recipes, MealPlans, Comments et Messages ;
- la suppression d'un Cookbook entraîne la suppression en cascade de ses CookbookMembers et Messages, mais met simplement à NULL le cookbookId des Recipes qu'il conte-

nait (`ON DELETE SET NULL`) : les recettes redeviennent personnelles plutôt que d’être perdues ;

- la suppression d’une `Recipe` entraîne la suppression en cascade de ses `Ingredients`, `Steps`, `Comments` et `MealPlans` associés.

5.4 Index et recherche

Outre les clés primaires et étrangères (indexées automatiquement par PostgreSQL), deux contraintes d’unicité indexées sont définies explicitement : `Users.email` et le couple (`provider`, `providerId`) de `OAuthAccounts`. La recherche plein texte (titre, ingrédients, tags) n’est pas réalisée via un index PostgreSQL dédié (par exemple `tsvector`/`GIN`) mais en mémoire côté serveur via `fuse.js` sur le résultat d’une requête filtrée, ce qui reste approprié au volume de données visé par l’application.

Chapitre 6

Vue d'ensemble de l'API REST

6.1 Conventions générales

Toutes les routes sont préfixées par `/api`. Sauf mention contraire, chaque route (hormis `/auth/register`, `/auth/login`, `/auth/google` et `/health`) requiert un en-tête `Authorization: Bearer <token>` validé par le middleware `requireAuth`. Les corps de requête et réponses sont au format JSON, à l'exception de l'upload d'image et d'import qui utilisent `multipart/form-data`.

Les ressources principales (cookbooks, recettes, plan de repas) suivent un schéma CRUD identique : `POST /` pour créer, `GET /` pour lister, `GET /:id` pour consulter, `PUT /:id` pour mettre à jour et `DELETE /:id` pour supprimer. Plutôt que de répéter ce schéma pour chaque ressource, le tableau ci-dessous en donne une vue consolidée (rôle minimum requis par opération); seules les routes qui s'écartent de ce modèle (authentification, sous-ressources, filtrage, import/export, WebSocket) sont détaillées dans les sections suivantes.

6.2 Vue d'ensemble des ressources

Ressource	Base	Créer	Lire	Modifier	Supprimer
Cookbooks	<code>/api/cookbooks</code>	authentifié	READER	EDITOR	CREATOR
Recettes	<code>/api/recipes</code>	authentifié (EDITOR si cookbook)	READER	EDITOR	EDITOR
Plan de repas	<code>/api/mealplan</code>	READER de la recette	authentifié	—	propriétaire

Créer une recette exige le rôle `EDITOR` du cookbook ciblé uniquement si elle est rattachée à un cookbook partagé; une recette personnelle (sans `cookbookId`) est libre pour tout utilisateur authentifié. Planifier un repas exige un accès en lecture à la recette planifiée (personnelle ou via le rôle `READER` du cookbook). Le plan de repas ne supporte pas de mise à jour : une entrée erronée doit être supprimée puis recréée.

6.3 Authentification — `/api/auth`

L'authentification ne suit pas le schéma CRUD ci-dessus ; ses routes sont détaillées intégralement.

Méthode	Route	Description
POST	<code>/register</code>	Création de compte (email + mot de passe), limité par rate limiting.
POST	<code>/login</code>	Connexion classique, retourne un JWT.
GET	<code>/me</code>	Profil de l'utilisateur authentifié.
PUT	<code>/me</code>	Mise à jour du profil (nom, avatar).
PUT	<code>/me/password</code>	Changement de mot de passe.
GET/PUT	<code>/me/preferences</code>	Lecture/écriture des préférences culinaires.
GET	<code>/google/config</code>	Expose l'identifiant client Google au front (si configuré).
POST	<code>/google</code>	Connexion/inscription via jeton d'identité Google.

6.4 Sous-ressources et actions spécifiques

Au-delà du CRUD standard, cookbooks et recettes exposent des sous-ressources propres. Les routes ci-dessous sont relatives à `/api/cookbooks/:id/` et `/api/recipes/:id/` respectivement.

Cookbooks — `/api/cookbooks/:id/`

Méthode	Sous-route	Rôle min.	Description
POST	<code>members</code>	CREATOR	Invitation d'un membre.
PUT	<code>members/:userId</code>	CREATOR	Changement de rôle d'un membre.
DELETE	<code>members/:userId</code>	CREATOR	Retrait d'un membre.
GET	<code>messages</code>	READER	Historique de la messagerie du cookbook.

Recettes — `/api/recipes/:id/`

Méthode	Sous-route	Rôle min.	Description
POST/DELETE	favorite	READER	Ajout/retrait des favoris.
POST	image	EDITOR	Upload de l'image, max 5 Mo.
GET	comments	READER	Liste des commentaires.
POST	comments	COMMENTER	Ajout d'un commentaire.
DELETE	comments/:commentId	READER	Suppression (auteur du commentaire ou CREATOR du cookbook).

6.5 Filtrage et recherche des recettes

L'endpoint GET `/api/recipes` accepte les paramètres de requête suivants, tous combinables :

Paramètre	Effet
q	Recherche plein texte (floue, via fuse.js).
searchIn	Restreint la recherche à une liste de champs parmi <code>title</code> , <code>tags</code> , <code>ingredients</code> .
cookbookId	Filtre par cookbook.
tags	Filtre par tags (liste séparée par des virgules).
ingredients	Filtre par ingrédients (liste séparée par des virgules).
maxPrepTime	Filtre par temps de préparation/cuisson maximum.
maxCookTime	
favorite	<code>true/false</code> — ne retourne que les recettes favorites de l'utilisateur.

6.6 Export et import — `/api/export`, `/api/import`

Méthode	Route	Description
GET	/api/export	Exporte les recettes personnelles et les cookbooks de l'utilisateur.
POST	/api/import	Importe un fichier (max 10 Mo) ; l'utilisateur devient créateur des recettes/cookbooks importés.

Les deux routes attendent un paramètre de requête `format` valant `json`, `csv` ou `mealie` (ex. `GET /api/export?format=json`). Trois formats sont supportés en export comme en import : **JSON** (format natif SUPMEAL, données en clair), **CSV** et un format **compatible Mealie**, répondant à l'exigence du sujet de permettre la migration ou le partage des données dans un format interprétable.

Outre la limite de taille de fichier (10 Mo, imposée par Multer), le contenu importé est borné après parsing à **500 recettes** et **50 cookbooks** au total (toutes recettes personnelles et de cookbooks confondues) ; un fichier dépassant l'une de ces limites est rejeté avec une erreur HTTP 400 explicite avant toute écriture en base.

6.7 Communication temps réel — Socket.io

Le serveur WebSocket (monté sur le même port que l'API, chemin `/socket.io/`) expose les événements suivants, après authentification par JWT dans le handshake (`socket.handshake.auth.token`)

Événement	Description
<code>cookbook:join</code>	Rejoint la room d'un cookbook (vérifie l'appartenance de l'utilisateur).
<code>cookbook:leave</code>	Quitte la room d'un cookbook.
<code>message:send</code>	Envoie un message (rôle COMMENTER minimum), persisté puis diffusé.
<code>message:new</code>	Événement serveur→client diffusé à tous les membres connectés de la room.

Chapitre 7

Informations nécessaires au fonctionnement

7.1 Variables d'environnement

L'ensemble de la configuration est porté par des variables d'environnement, centralisées dans un fichier `.env` à la racine du projet et chargées à la fois par `docker-compose.yml` et par le serveur (`dotenv`). Un fichier `.env.example` documenté est fourni à la racine du dépôt à titre de modèle ; **aucun secret réel n'est commité**.

Variable	Exemple / défaut	Description
POSTGRES_USER	supmeal	Utilisateur PostgreSQL créé au premier démarrage du conteneur db.
POSTGRES_PASSWORD	—	Mot de passe PostgreSQL. À redéfinir obligatoirement en production.
POSTGRES_DB	supmeal	Nom de la base créée au premier démarrage.
DATABASE_URL	postgresql://user:pass@db:5432/db	Chaîne de connexion complète utilisée par Sequelize côté serveur.
JWT_SECRET	—	Clé secrète de signature des JWT. À générer aléatoirement et à garder confidentielle.
JWT_EXPIRES_IN	7d	Durée de validité d'un jeton émis.
PORT	4000	Port d'écoute interne du serveur Express.
NODE_ENV	development production	Environnement d'exécution Node.js.

Variable	Exemple / défaut	Description
GOOGLE_CLIENT_ID	—	Identifiant client OAuth2 Google. Laissé vide, la connexion Google est désactivée proprement (HTTP 501).
CLIENT_URL	http://localhost:3000	Origine du client, utilisée pour la configuration CORS et Socket.io.
SERVER_URL	http://localhost:4000	URL publique du serveur API.

7.2 Prérequis pour activer la connexion Google

La connexion via Google est désactivée par défaut (le contrôleur retourne alors une erreur HTTP 501 explicite). Pour l'activer :

1. Créer un projet dans la [Google Cloud Console](#).
2. Configurer un identifiant OAuth 2.0 de type *application web*, avec l'origine du client (CLIENT_URL) autorisée.
3. Renseigner l'identifiant client obtenu dans la variable GOOGLE_CLIENT_ID.

7.3 Prérequis logiciels

Mode d'exécution	Prérequis
Via Docker Compose (recommandé)	Docker Engine ≥ 24 , Docker Compose v2
Développement local sans Docker	Node.js ≥ 20 , PostgreSQL ≥ 16

7.4 Secrets et rendu du projet

Conformément au cahier des charges, aucun secret (mot de passe, clé JWT, identifiant OAuth) ne doit être présent en clair dans le dépôt livré. Le fichier `.env` contenant les valeurs réelles est exclu du contrôle de version via `.gitignore`; seul `.env.example`, contenant des valeurs de démonstration non sensibles, est versionné.

Chapitre 8

Guide de déploiement

8.1 Déploiement via Docker Compose (recommandé)

C'est le mode de déploiement de référence de l'application : trois services distincts (client, server, db) sont orchestrés par le fichier `docker-compose.yml` situé à la racine du dépôt.

8.1.1 Étapes

1. Cloner le dépôt et se placer à sa racine.
2. Copier le fichier d'exemple et renseigner les valeurs :

```
1 cp .env.example .env
2 # Éditer .env : définir POSTGRES_PASSWORD et JWT_SECRET
3 # avec des valeurs uniques et non triviales.
```

3. Construire les images et lancer l'ensemble des services :

```
1 docker compose up --build -d
```

4. Vérifier que les trois conteneurs sont démarrés et sains :

```
1 docker compose ps
```

5. L'application est accessible sur <http://localhost:3000> (client) ; l'API est joignable directement sur <http://localhost:4000/api> à des fins de débogage.

8.1.2 Détail des services

Service	Image / build	Détails
db	postgres:16-alpine	Volume nommé <code>postgres_data</code> pour la persistance; healthcheck via <code>pg_isready</code> .
server	build ./server	Dépend de la santé de db (<code>depends_on.condition: service_healthy</code>); volume <code>uploads_data</code> pour les images; port 4000 exposé.

Service	Image / build	Détails
client	build ./client (Nginx)	Dépend de server; sert les fichiers statiques et reverse-proxy /api, /uploads et /socket.io; port 80 interne mappé sur 3000.

Au démarrage, le serveur exécute `sequelize.sync()` : le schéma de base de données (tables, contraintes, index) est créé automatiquement s'il n'existe pas encore — aucune étape de migration manuelle n'est requise pour un premier déploiement.

8.1.3 Arrêt et nettoyage

```
1 docker compose down          # Arrête les conteneurs (conserve les volumes)
2 docker compose down -v       # Arrête et supprime les volumes (perte des données)
```

8.2 Déploiement en environnement de développement local (sans Docker)

Ce mode est utile pour le développement itératif avec rechargement à chaud.

8.2.1 Base de données

Installer et démarrer PostgreSQL 16 localement, puis créer la base et l'utilisateur correspondant aux valeurs de `.env` (ou démarrer uniquement le service db via `docker compose up db -d` tout en exécutant client et serveur en local).

8.2.2 Serveur

```
1 cd server
2 npm install
3 npm run dev          # nodemon, rechargement à chaud sur le port 4000
```

8.2.3 Client

Le client étant entièrement statique, tout serveur de fichiers statiques convient (ex. l'extension *Live Server* de VS Code, ou `npx serve client`). Il est nécessaire que les appels vers `/api`, `/uploads` et `/socket.io` soient redirigés vers le serveur (port 4000), à l'identique de la configuration Nginx de production (voir `client/nginx.conf`) — à défaut, adapter l'URL de base dans `client/js/api.js`.

8.3 Vérification post-déploiement

- GET /api/health doit retourner {"status": "ok"}.
- La création d'un compte puis d'une recette permet de valider la chaîne complète client → serveur → base de données.
- L'ouverture de deux sessions dans un même cookbook permet de valider la messagerie instantanée (Socket.io).

Chapitre 9

Sécurité

9.1 Authentification et gestion des secrets

- Les mots de passe ne sont jamais stockés en clair : ils sont hachés avec `bcrypt.js` (facteur de coût 10) avant toute persistance.
- La comparaison lors de la connexion utilise systématiquement `bcrypt.compare` contre un hash factice (`DUMMY_PASSWORD_HASH`) lorsque l'utilisateur n'existe pas, afin d'éviter une attaque par mesure du temps de réponse permettant de deviner si un email est enregistré.
- Les jetons JWT sont signés avec un secret (`JWT_SECRET`) qui ne doit exister qu'en variable d'environnement, jamais dans le code source ou le dépôt Git.
- Les jetons Google sont vérifiés côté serveur (`verifyIdToken`), et l'application s'assure que l'email associé est vérifié par Google (`email_verified`) avant de créer ou de lier un compte.

9.2 Protection HTTP

- **Helmet** applique un ensemble d'en-têtes HTTP de sécurité par défaut (protection XSS basique, désactivation du sniffing MIME, etc.).
- **CORS** est restreint à la seule origine du client (`CLIENT_URL`), avec support des identifiants (credentials).
- **express-rate-limit** limite les tentatives de connexion/inscription/authentification Google à 10 requêtes par IP toutes les 15 minutes, avec une instance dédiée par route pour éviter qu'un utilisateur bloqué sur une route ne le soit également sur les autres.

9.3 Contrôle d'accès et permissions

Le modèle de rôles à quatre niveaux (`CREATOR > EDITOR > COMMENTER > READER`) demandé par le cahier des charges est appliqué systématiquement côté serveur via les middlewares `requireCookbookRole`, `requireRecipeAccess` et `requireCommentAccess`, qui interrogent l'appartenance réelle de l'utilisateur avant chaque action sensible. Le client ne fait qu'afficher ou masquer des éléments d'interface en fonction du rôle retourné par l'API : cette information n'est **jamais** la source d'autorité de la décision d'autoriser une action, uniquement du confort d'affichage.

9.4 Validation des entrées

Toutes les routes recevant des données utilisateur (corps de requête, paramètres d'URL, paramètres de requête) sont validées via `express-validator` avant d'atteindre le contrôleur, avec des règles explicites de type, de longueur et de format (ex. UUID pour les identifiants). Les erreurs de validation sont centralisées par le middleware `handleValidationErrors` et retournées sous forme de réponse HTTP 400 structurée.

9.5 Upload de fichiers

- Les images de recettes sont restreintes par type MIME (`jpeg`, `png`, `webp`, `gif`) et par taille (5 Mo maximum) ; le nom de fichier sur disque est généré via `uuid` plutôt que dérivé du nom fourni par le client, ce qui écarte les risques de traversal de chemin ou de collision.
- Les fichiers d'import (JSON/CSV/Mealie) sont limités à 10 Mo et traités en mémoire, sans jamais être écrits sur disque ni interprétés comme du code exécutable.

9.6 Isolation réseau

Dans le déploiement Docker Compose, la base de données n'est accessible que depuis le réseau interne créé par Compose ; en production, il est recommandé de ne pas publier le port 5432 sur l'hôte (suppression de la section `ports` du service `db`) afin de ne pas exposer PostgreSQL directement sur Internet.